CONCURRENTPROCESSANDPROGRAMMING LECTURE 1

PROCESSSANDTHREADS

Processes

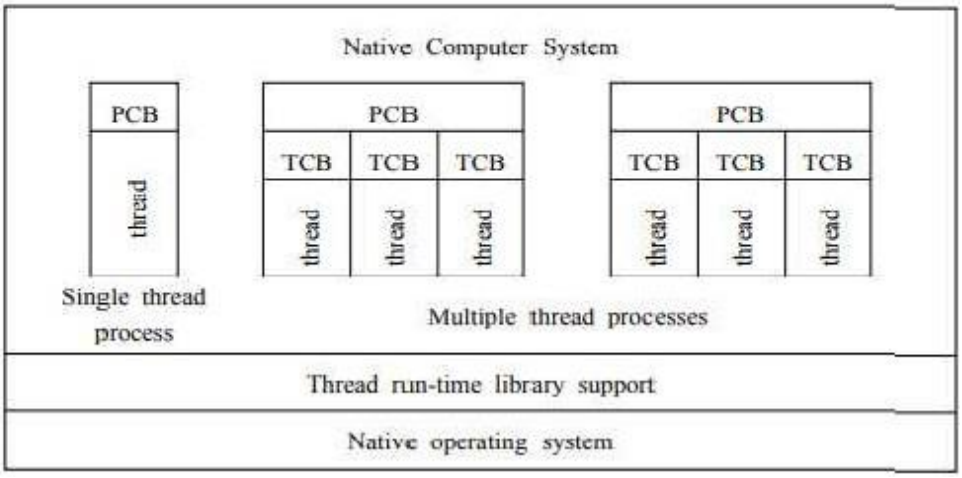
- The chief task of an operating system is to manage a set of processes and to perform work on their behalf.
- So far we have considered processes as independent. They interact in some waywith the operating system, but we have not examined ways in which processes can interact with each other.
- Processesthat interactwitheachby**cooperating** toachievea**commongoal**are called **concurrent processes**.
  - Heavyweightprocesses(processes)
  - Lightweightprocesses(threads)
- Manyproblemscanbenaturallyviewedinawaythatleadstoconcurrentprogramming.

Thread

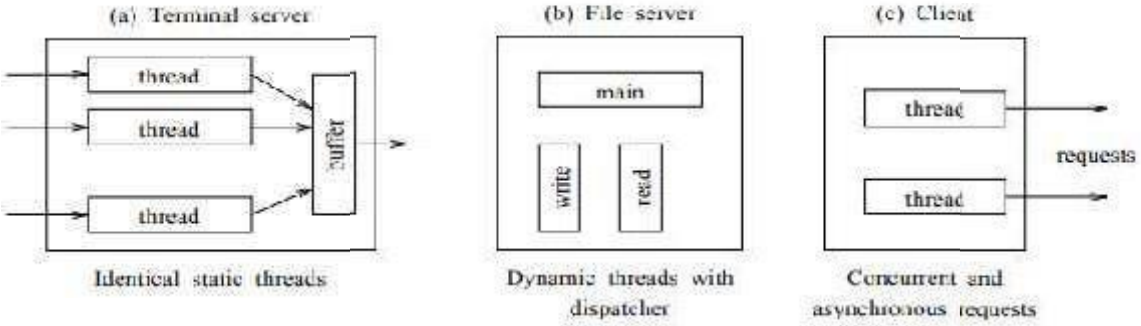
In computer science,a thread of execution is the smallestsequence of programmed instructions that can be managed independently by a scheduler, which is typically a part of the operating system. Theimplementation ofthreadsand processes differs between operating systems,butin most cases a thread is a component of a process. Multiple threads can exist within one process, executing concurrently and sharing resources such as memory, while different processes do not share these resources. In particular, the threads of a process share its executable code and the values of its dynamically allocated variables and non-thread-local global variables at any given time.

- **Processes:**separatelogicaladdressspace
- **Threads:**commonlogicaladdressspace

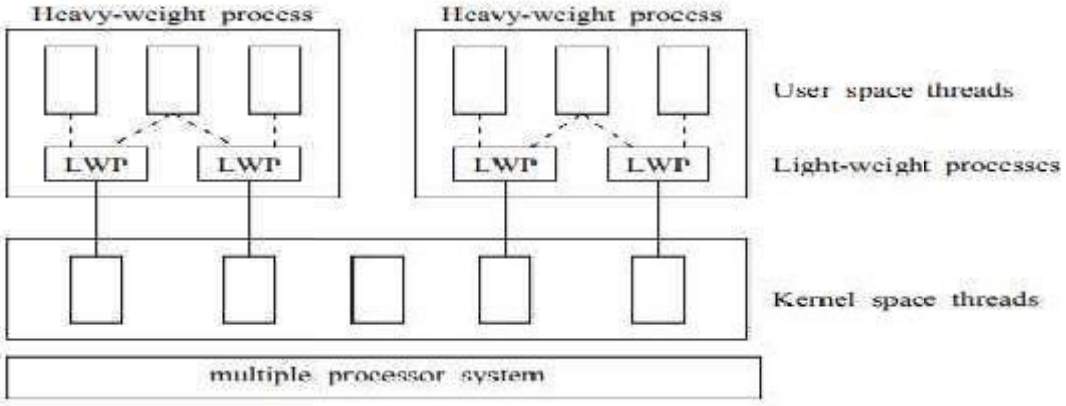
Two-level concurrency of processes and threads



Thread applications



Solaris thread implementation

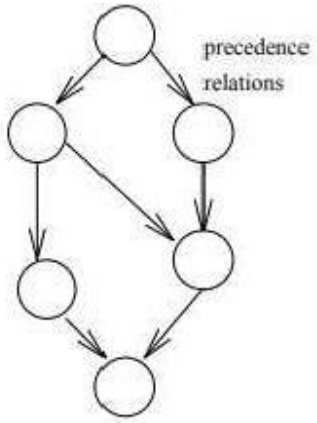


GraphModelforprocessrepresentation:

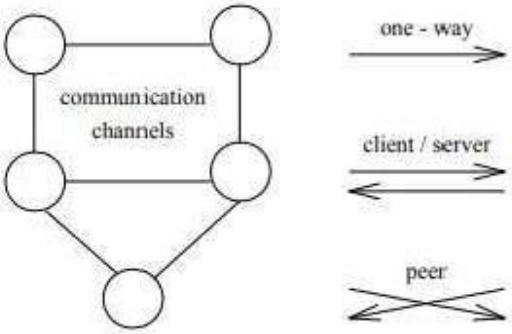
Processesarerelatedbytheirneedforsynchronizationand/orcommunication.

Aspecialcaseofsynchronizationistheprecedencerelationshipbetweenprocesses.

Graph representations



Synchronous process graph



Asynchronous process graph and communication scenarios

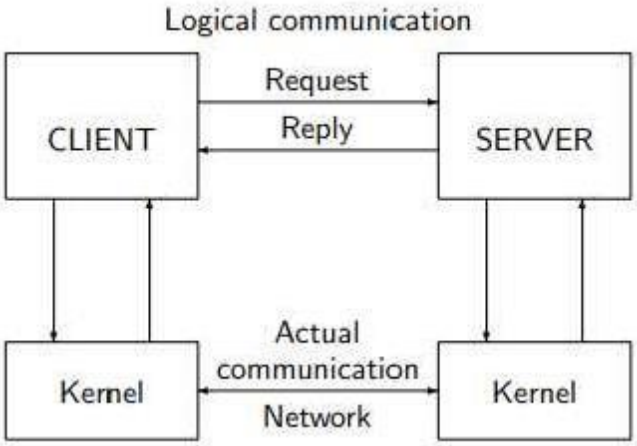
LECTURE2

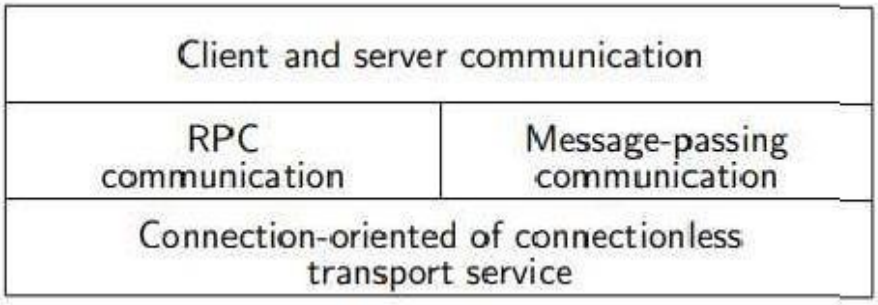
ClientServerModel:

• Interactionthroughasequenceofrequestsandresponses.

• Service-orientedcommunicationmodel.

• Higher-levelabstractionofIPCthanRPCormessagepassingcommunication.





Timeservices

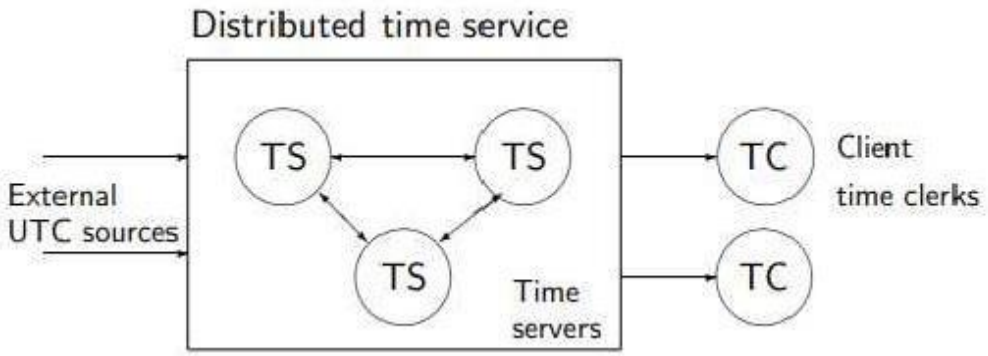
• Clocks are used to represent time (a relative measure of a point of time) and timer (an absolute measure of a time interval); it is used to describe the occurence of events in three different ways: – When an event occurs – How long it takes – Which event occurs first

• Thereisnoglobaltimeindistributedsystems

• Physicalclock:closeapproximationofrealtime(bothpointandinterval)

- Logicalclock:preserversonlytheorderingofevents

Physicalclocks:



- Compensatingdelay:
  - UTCsourcestotimeservers
  - timeserverstoclients
- Calibrating discrepancy

  Applicationofphysicalclocks:
- Protocolsrelyonatime-outforhandlingexceptions
- Timestampingforsecureinternetcommunication(avoidingplaybackattacks)

Logicalclocks

For many applications, events need not be scheduled or synchronized with respect to the real-time clock; it is only the ordering of event execution that is of concern. **Lamport's logical clock** is a fundamental concept for ordering of processes and events in distributed systems.

- EachprocessPiinthesystemmaintainsalogicalclockCi
- →:happens-beforerelationtosynchronizethelogicalclock
- a→b:Eventaprecedesevent b
- Withinaprocess,ifeventaprecedeseventb,thenC(a)<C(b)

- Thelogical clock ina processisalways incremented by anarbitrarypositive numberwhen events in the process progress
- Processesinteractwitheachotherusingapairofsendandreceiveoperations:theseare considered events as well Rules for Lamport's logical clock.

LECTURE3

RulesforLamport'slogicalclock:

1. ifa→bwithinthesameprocessthenC(a)<C(b)

2. IfaisthesendingeventofPiandbisthecorrespondingreceivingeventofPjthen Ci(a) < Cj(b) (can be enforced if the sending process timestamps its logical clock in the message and receiving process updates its logical clock using the larger of its own clock time and the incremented time stamp)

Implementationoftherules:

1. C(b)=C(a)+d

2. Cj(b)=max(TSa+d,Cj(b)),TSaisthetimestampofthesendingevent

The happens-before relation describes the causality between two events; it is transitive.

Twoevents,aandb,aresaidtobedisjointevents andcanberunconcurrently ifneithera→b nor b →a.

Rules1and2resultinpartialordering,soathirdrulecanbe added:

3Foralleventsaandb,C(a)6=C(b)(thismustbefulfilledonlyfordisjointevents)

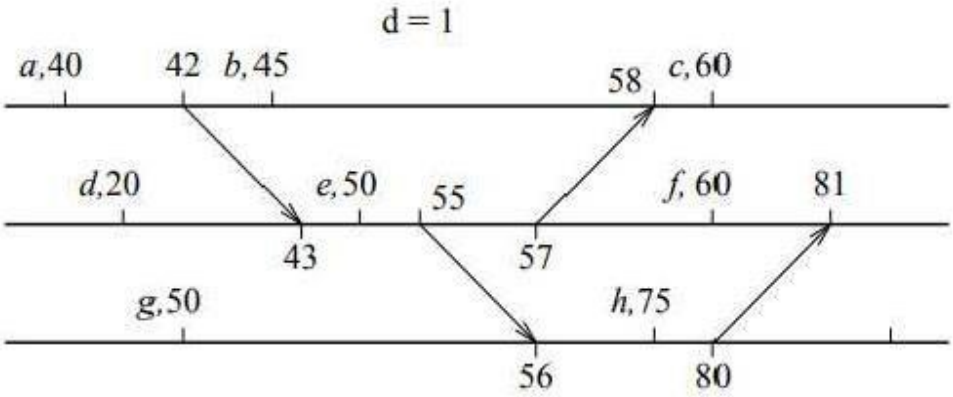
System-wide unique logical clock times forall events can be obtained by concatenating the logical clock with a distinct process ID number.

The happens-before relation has an important property: Ci(a) < Cj(b) doesn't imply a → b, i.e. we cannot distinguish disjoint events using values of logical clocks.

EXAMPLE:-

Implementation:

$C(b) = C(a) + d$ and
$C_j(b) = max(TS_a + d, C_j(b))$,
where TS_a is the timestamp of the sending event and d is a positive number.



So, $a \rightarrow b \implies C(a) < C(b)$, but $C(a) < C(b) \not\Rightarrow a \rightarrow b$.

LECTURE4

LANGUAGE MECHANISMS FOR SYNCHRONIZATION

Concurrentlanguages

- Specificationofconcurrentactivities
- Synchronizationofprocesses
- Interprocesscommunication
- Nonderterministicexecutionofprocesses

Languageconstructs

- Programstructure
- Datastructure
- Controlstructure
- Procedureandsystemcall
- Inputandoutput

Synchronizationmechanismsandlanguagefacilities

Synchronization Methods	*Language Facilities*
Shared-Variable Synchronization	
semaphore	shared variable and system call
monitor	data type abstraction
conditional critical region	control structure
serializer	data type and control structure
path expression	data type and program structure
Message Passing Synchronization	
communicating sequential processes	input and output
remote procedure call	procedure call
rendezvous	procedure call and communication

Shared-variablesynchronization

- Semaphoreandconditionalcriticalregion
- Monitorandserializer
- Pathexpression

ClassicProblems

- CriticalSection
- Dining Philosophers
- Readers/Writers
- Producer-Consumer

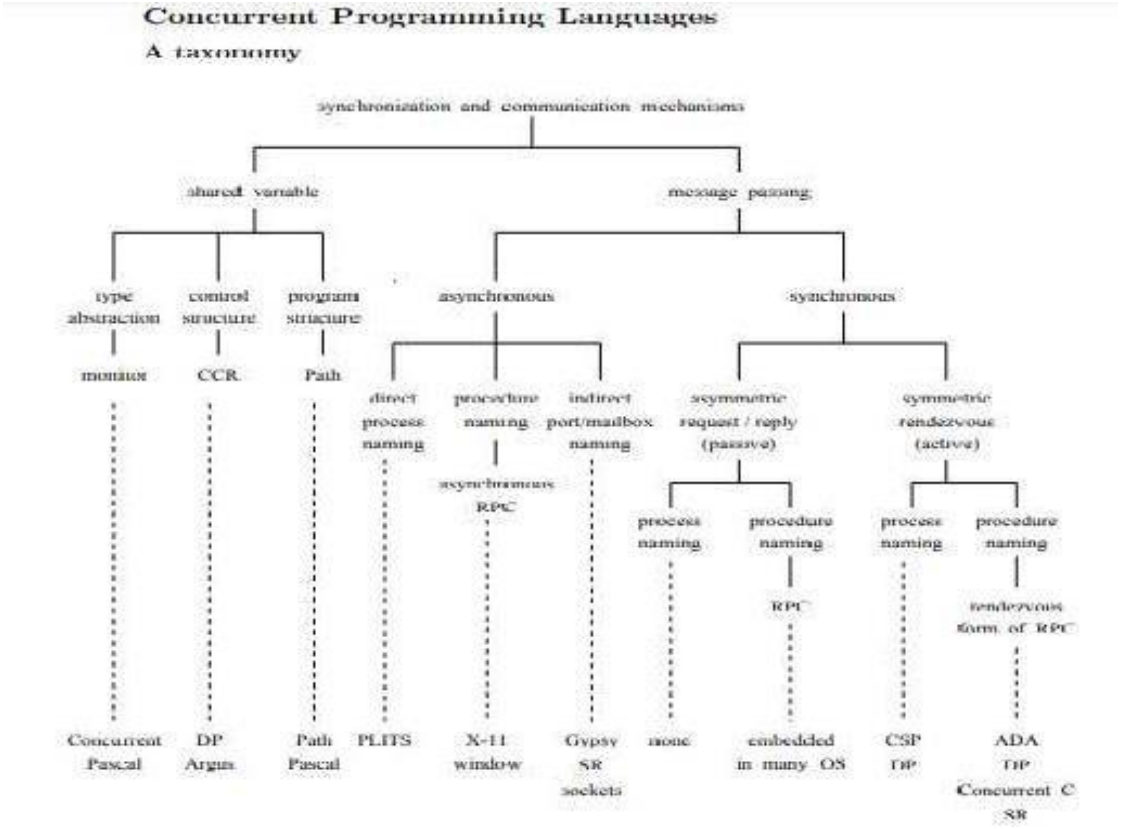
Example:theReader/WriterProblems

synchronization + concurrency

Basics

- ifDBempty,allowanyone in
- ifreaderinDB,writernotallowedin
- ifwriterinDB,nobodyallowedin

Lock Requested	Lock Held	
	Read Lock	Write Lock
Read Lock	✓	✗
Write Lock	✗	✗


Concurrent Programming Languages
A taxonomy
synchronization and communication mechanisms
shared variable
message passing
type abstraction
control structure
program structure
asynchronous
synchronous
monitor
CCR
Path
direct process naming
procedure naming
indirect port/mailbox naming
symmetric request / reply (passive)
symmetric rendezvous (active)
asynchronous RPC
process naming
procedure naming
process naming
procedure naming
RPC
rendezvous form of RPC
Concurrent Pascal
DP Argus
Path Pascal
PLITS
X-11 window
Gypsy SR sockets
none
embedded in many OS
CSP DP
ADA DP Concurrent C SR

LECTURE5

INTER-PROCESSCOMMUNICATION

MessagePassingSynchronization

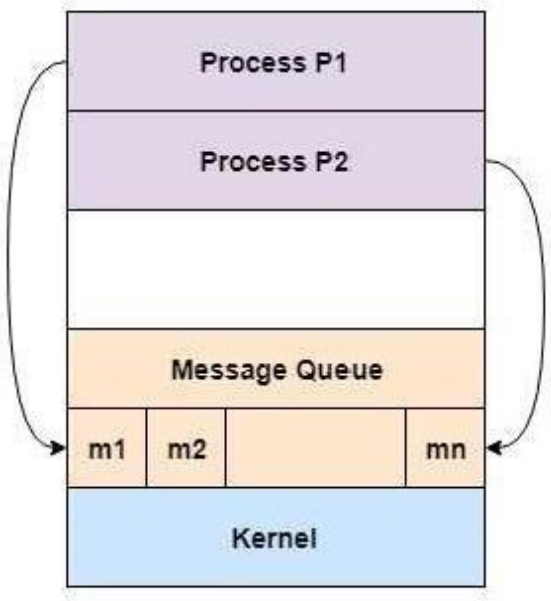
- **Asynchronous:**non-blockingsend,blockingreceive
- **Synchronous:**blockingsend,blockingreceive

Messagepassing

Process communicationisthemechanismprovidedbytheoperatingsystemthatallowsprocesses tocommunicate witheach other. This communication could involve a process letting anotherprocess knowthat someeventhas occurredortransferringofdatafromoneprocessto another. One of the models of process communication is the message passing model.

Message passing modelallows multiple processestoreadandwrite data tothe message queue without being connected to each other. Messages are stored on the queue until their recipient retrieves them. Message queues are quite useful for interprocess communication and are used by most operating systems.

A diagram that demonstrates message passing model of process communication is given as follows −



Message Passing Model

AdvantagesofMessagePassingModel

Someoftheadvantagesofmessagepassingmodelaregivenasfollows−

- Themessagepassingmodelismucheasiertoimplementthanthesharedmemory model.
- It is easiertobuildparallelhardware using message passing modelas it is quite tolerant of higher communication latencies.

DisadvantageofMessagePassingModel

The message passing model has slower communication than the shared memory model because the connection setup takes time.

Request/ReplyandTransactionCommunication

Asynchronous:

sendercontinuesaftersubmission

Synchronous:

senderisblockeduntil

– messageisstoredatreceiver's host

– messageisreceived

– replyisreceived

Basicallyallclient/servercommunicationfollowsthepatternofa **request/replyprotocol**:

– theclientsendsarequestmessage

– theserverexecutestherequestedoperation

– theserverrespondswithareplymessage

NameandDirectoryServices Naming

• In a distributed system, names are used to refer to a wide variety of resources (computers,services, remote objects and files, ...).

• Namesfacilitatecommunicationandresourcesharing.

• Processes cannot share particular resources managed by a computer system unless they can name them consistently.

• Users cannot communicate with one another via a distributed system unless they can name one another, for example, with email addresses.

NameServices

• Anameservicestoresacollectionofoneormorenaming contexts.

• Naming contexts: sets of bindings between textual names and attributes for objects (such as users, computers, services and remote objects).

• Majoroperationofanameservice:name resolution,that istolook upattributesfromagiven name.

Example:

DomainNameSystem(DNS)namesobjectsacrosstheInternet**(mapshuman-friendly hostnames into IP addresses)**.

DirectoryServices

Directory service: a service that stores collections of bindings between names and attributesand that looks up entries that match attribute-based specifications.

A directory service returns the sets of attributes of any objects found to match some specified attributes.

Example:

therequest“TelephoneNumber=020-5559980”mightreturn{“Name=JohnSmith”, “TelephoneNumber = 020-5559980”, ...}

LECTURE6

RPCANDRMICASE STUDIES:

RemoteProcedureCalls(RPC)

Although message passing can be effective, it would be nice if there were a more uniform, reusable, and user-friendly way of doing things. **RemoteProcedure Calls (RPCs) provide such an abstraction.** Instead of viewing the communication between two systems in terms of independent exchanges of data, we back up a step and examine the overall behaviors of the systems.

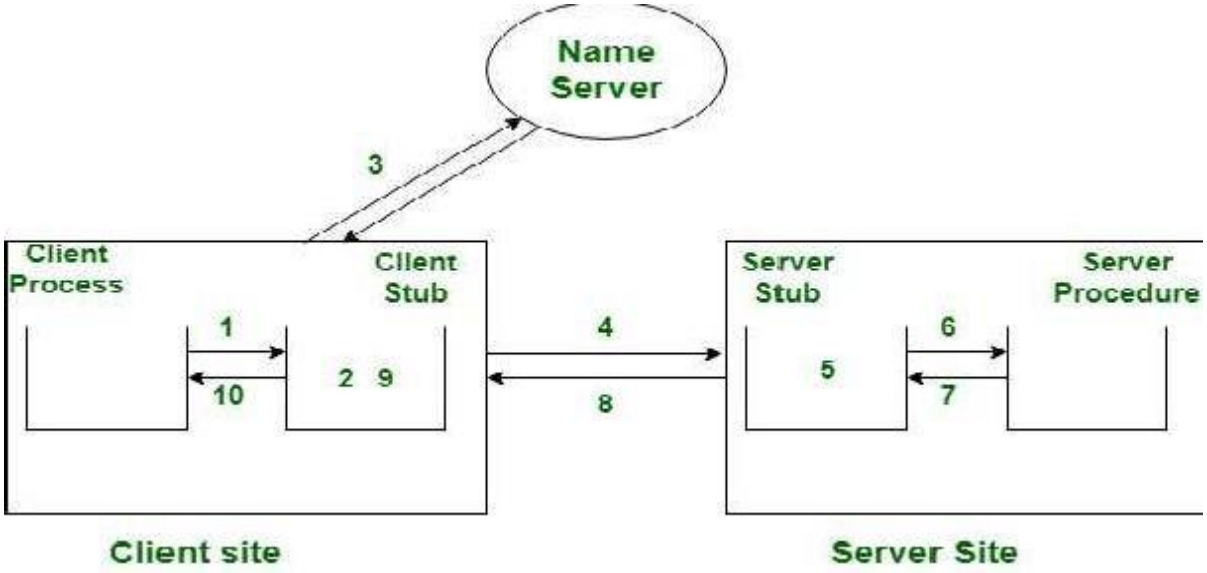
Often times the services of the system can often be decomposed into procedures, much like those used in traditional programming. These procedures accept certain types of information, perform some useful operation, and then return a result. The RPC abstraction allows us to extend this paradigm to distributed systems. One system can provide remote procedure callsfor use by other systems. From the applications point of view, it can use these remote procedure calls much like local procedure calls. Behind the scences, the **RPC is actually connecting to a remote host, sending it the parameters, performing an operation on that remote host, and then returning the result.**

This is very similar to a very specific use of message passing. In fact the function invocation is a message from the client to the server. This message names the function and also provides the parameters. After receiving this message, the server performs the operation and sends a messagebacktotheclientwitha result. The clientthentreatsthis resultas if itwere the return value from a local procedure call.

TheimportantcharacteristicsofRPCsarethese:

- Theyprovideaveryfamiliarinterface fortheapplicationdeveloper
- Theyonewayofimplementingthecommonplacerequest-replyprimitive
- Theformatofthemessagesisstandard,notapplicationdependent

- Theymakeiteasiertoreusecode,sincetheRPCshaveastandardinterfaceandare separate from any one application-proper.



TheabovediagramshowstheworkingstepsinPRCimplementation.

LECTURE7

RMI(RemoteMethodInvocation)

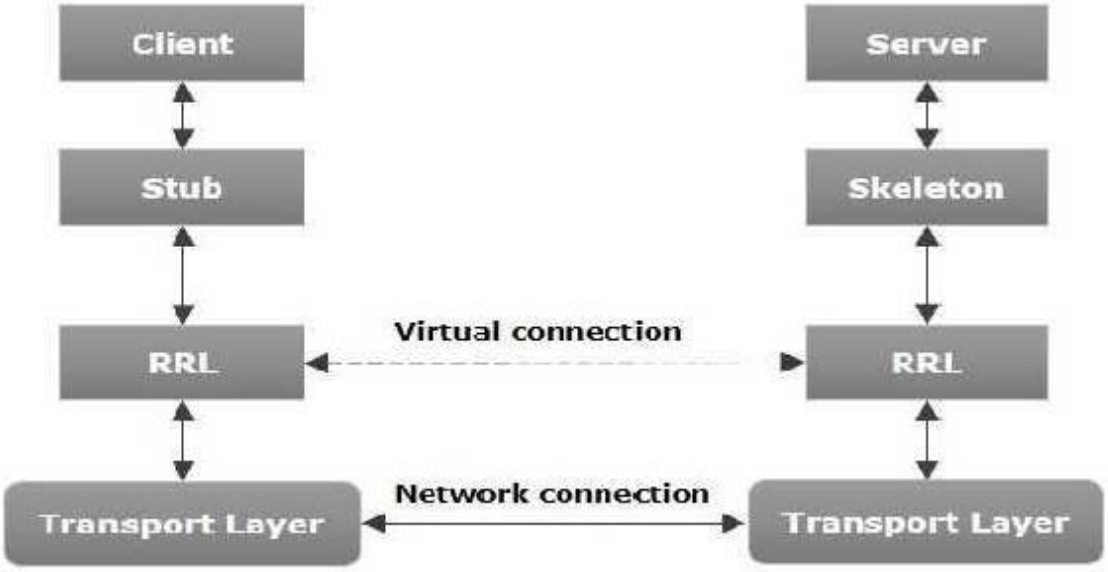
- RMIstandsfor **RemoteMethodInvocation**.Itisamechanismthatallowsan objectresidinginone system(JVM)toaccess/invokeanobjectrunningon another JVM.
- RMI is used to build distributed applications; it provides remote communication between Java programs. It is provided in the package **java.rmi**.

ArchitectureofanRMIApplication

InanRMIapplication,wewritetwoprograms,a **serverprogram** (residesontheserver)and a **client program** (resides on the client).

- Insidetheserverprogram,aremoteobjectiscreatedandreferenceofthatobjectis made available for the client (using the registry).
- Theclientprogramrequeststheremoteobjectsontheserverandtriestoinvokeits methods.

The following diagram shows the architecture of an RMI application.



WorkingofanRMIApplication

ThefollowingpointssummarizehowanRMIapplicationworks−

- Whentheclientmakesacalltotheremoteobject,itisreceivedbythestubwhich eventually passes this request to the RRL.
- When theclient-sideRRLreceivestherequest,itinvokesa methodcalled **invoke()**of the object **remoteRef**. It passes the request to the RRL on the server side.
- TheRRLontheserversidepassestherequesttotheSkeleton(proxyontheserver) which finally invokes the required object on the server.
- Theresultispassedallthewaybacktotheclient.

Let'sseethatthedifferencebetweenRPCandRMI:

S.NO	RPC	RMI
1.	RPCisalibraryandOSdependent platform.	Whereasitisajavaplatform.
2.	RPCsupportsprocedural programming.	RMIsupportsobjectoriented programming.
3.	RPCislessefficientincomparisonof RMI.	WhileRMIismoreefficientthanRPC.
4.	RPCcreatesmoreoverhead.	While it creates lessoverhead than RPC.
5.	Theparameterswhicharepassedin RPCareordinaryornormaldata.	While in RMI, objects are passed as parameter.
6.	RPCistheolderversionofRMI.	WhileitisthesuccessorversionofRPC.
7.	ThereishighProvisionofeaseof	WhilethereislowProvisionofeaseof

S.NO	RPC	RMI
	programminginRPC.	programminginRMI.
8.	RPCdoesnotprovideanysecurity.	Whileitprovidesclientlevelsecurity.
9.	It'sdevelopmentcostishuge.	While it'sdevelopment cost is fair or reasonable.
10.	Thereisahugeproblemof versioning in RPC.	Whilethere ispossible versioning using RDMI.
11.	Thereis multiple codesare needed for simple application in RPC.	While there is multiple codes are not needed forsimple application in RMI.

MarshallingandUnmarshalling:

Whenever a client invokes a method that accepts parameters on a remote object, the parameters are bundled into a message before being sent over the network. These parameters may be of primitive type or objects. In case of primitive type, the parameters are put together and a header is attached to it. In case the parameters are objects, then they are serialized. This process is known as **marshalling**.

At the server side, the packed parameters are unbundled and then the required method is invoked. This process is known as **Unmarshalling**.